

SQL Operators & Scalar Functions – Quick Glance Guide

This document condenses **Lab 04** into a **clean, exam-ready reference**. Each item includes **keyword/function**, **what it does**, **syntax**, and a **simple example**.

1. Arithmetic Operators

Operator	Description	Syntax	Example
<code>+</code>	Addition	<code>expr + expr</code>	<code>marks + 5</code>
<code>-</code>	Subtraction	<code>expr - expr</code>	<code>salary - 1000</code>
<code>*</code>	Multiplication	<code>expr * expr</code>	<code>qty * price</code>
<code>/</code>	Division	<code>expr / expr</code>	<code>total / 4</code>
<code>%</code> / <code>MOD()</code>	Remainder	<code>MOD(x, y)</code>	<code>MOD(10, 3) → 1</code>

Example:

```
SELECT marks + 5 AS updated_marks FROM Student;
```

2. Logical & Conditional Operators

AND / OR / NOT

Operator	Purpose	Syntax
AND	All conditions true	<code>WHERE c1 AND c2</code>
OR	Any condition true	<code>WHERE c1 OR c2</code>
NOT	Negates condition	<code>WHERE NOT c1</code>

```
SELECT * FROM employees WHERE dept_id = 10 AND salary > 50000;
```

BETWEEN

Selects values **within an inclusive range**.

Syntax:

```
WHERE column BETWEEN low AND high
```

Example:

```
SELECT * FROM employees WHERE salary BETWEEN 30000 AND 60000;
```

IN

Shorthand for multiple OR conditions.

Syntax:

```
WHERE column IN (v1, v2, v3)
```

Example:

```
SELECT * FROM employees WHERE department_id IN (10, 20, 30);
```

LIKE (Pattern Matching)

Wildcard	Meaning
%	Any number of characters
_	Exactly one character

Syntax:

```
WHERE column LIKE pattern
```

Example:

```
SELECT * FROM employees WHERE first_name LIKE 'E1%';
```

3. Operator Precedence (High → Low)

1. **()** Parentheses
2. Arithmetic *** / + -**
3. Comparison **= < > BETWEEN IN LIKE**

- 4. NOT
- 5. AND
- 6. OR

4. Common SQL Keywords

DISTINCT

Returns **unique values only**.

```
SELECT DISTINCT department_id FROM employees;
```

ALIAS (AS)

Temporary rename for readability.

```
SELECT salary AS monthly_salary FROM employees;
```

ORDER BY

Sorts result set.

```
SELECT * FROM employees ORDER BY salary DESC;
```

TOP / LIMIT / ROWNUM / FETCH

Used to limit rows returned.

Oracle (ROWNUM):

```
SELECT * FROM employees WHERE ROWNUM <= 5;
```

Oracle 12c+:

```
SELECT * FROM employees ORDER BY employee_id FETCH FIRST 5 ROWS ONLY;
```

OFFSET (Pagination):

```
SELECT * FROM employees ORDER BY employee_id OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY;
```

5. SQL Functions Overview

Type	Operates On	Output
Scalar	Single row	One result per row
Aggregate	Multiple rows	One result per group

6. Scalar (Single-Row) Functions

Character Functions

Function	Description	Syntax	Example
<code>LOWER()</code>	Convert to lowercase	<code>LOWER(string)</code>	<code>LOWER('SQL')</code> → sql
<code>UPPER()</code>	Convert to uppercase	<code>UPPER(string)</code>	<code>UPPER('sql')</code> → SQL
<code>TRIM()</code>	Remove leading/trailing spaces	<code>TRIM(string)</code>	<code>TRIM(' hi ')</code>
<code>LENGTH()</code>	String length	<code>LENGTH(string)</code>	<code>LENGTH('FAST')</code> → 4
<code>CONCAT()</code>	Join strings	<code>CONCAT(e1, e2, ...)</code>	<code>CONCAT('A', 'B')</code>
<code>REPLACE()</code>	Replace substring	<code>REPLACE(str, old, new)</code>	<code>REPLACE('abc', 'a', 'z')</code>
<code>SUBSTR()</code>	Extract substring	<code>SUBSTR(str, start, len)</code>	<code>SUBSTR('Oracle', 1, 3)</code>
<code>INSTR()</code>	Find position	<code>INSTR(str, substr)</code>	<code>INSTR('Oracle', 'a')</code>

Numeric Functions

Function	Description	Syntax	Example
<code>MOD()</code>	Remainder	<code>MOD(x, y)</code>	<code>MOD(10, 3)</code> → 1
<code>ROUND()</code>	Round value	<code>ROUND(num, d)</code>	<code>ROUND(12.456, 2)</code>
<code>TRUNC()</code>	Truncate decimals	<code>TRUNC(num, d)</code>	<code>TRUNC(12.456, 2)</code>
<code>CEIL()</code>	Smallest integer $\geq n$	<code>CEIL(num)</code>	<code>CEIL(4.2)</code> → 5

Date Functions

Function	Purpose	Syntax	Example
Date subtraction	Days difference	<code>date2 - date1</code>	<code>SYSDATE - hire_date</code>
<code>ADD_MONTHS()</code>	Add months	<code>ADD_MONTHS(date, n)</code>	<code>ADD_MONTHS(SYSDATE, 2)</code>
<code>CURRENT_DATE</code>	Current date	<code>CURRENT_DATE</code>	<code>SELECT CURRENT_DATE FROM dual;</code>
<code>CURRENT_TIME</code>	Current time	<code>CURRENT_TIME</code>	<code>SELECT CURRENT_TIME FROM dual;</code>
<code>CURRENT_TIMESTAMP</code>	Date & time	<code>CURRENT_TIMESTAMP</code>	<code>SELECT CURRENT_TIMESTAMP FROM dual;</code>
<code>TRUNC()</code>	Remove time	<code>TRUNC(date)</code>	<code>TRUNC(SYSDATE)</code>

Conversion Functions

Function	Use	Syntax	Example
<code>TO_CHAR()</code>	Format date/number	<code>TO_CHAR(val, format)</code>	<code>TO_CHAR(SYSDATE, 'DD-MON-YYYY')</code>
<code>CAST()</code>	Change datatype	<code>CAST(val AS type)</code>	<code>CAST('2500' AS INTEGER)</code>

NULL Handling (General Functions)

Function	Purpose	Syntax	Example
<code>NULLIF()</code>	NULL if equal	<code>NULLIF(a, b)</code>	<code>NULLIF(10, 10)</code>
<code>COALESCE()</code>	First non-NULL	<code>COALESCE(v1, v2, ...)</code>	<code>COALESCE(NULL, 25, 50)</code>
<code>NVL()</code>	Replace NULL	<code>NVL(expr, val)</code>	<code>NVL(NULL, 100)</code>

Final Tip (Exam-Friendly)

- Use **BETWEEN / IN** instead of long **OR** chains
- Remember precedence: **AND** before **OR**
- **Scalar** → row-wise, **Aggregate** → group-wise

End of Quick Reference 🚀